Jdev ERP Pro
Relatório da sessão de 11/11/2026

O que foi feito no back-end, como foi verificado e o que ficou registrado para depois

---

Módulo: `jdev-erp-pro-back-end-dev` (o módulo `-pro` não foi alterado)

Estado: todas as mudanças estão no working tree, sem commit

Verificação final: `mvn -Dmaven.compiler.proc=full clean test` → 337 testes, 0 falhas, BUILD SUCCESS (cerca de 45 s)

Stack: Java 25 · Spring Boot 4.0.6 · Hibernate 7 · Spring Security + JWT · PostgreSQL 18

Resumo

Nesta sessão, o back-end passou a devolver DTO em todos os endpoints. Também passou a funcionar com `spring.jpa.open-in-view=false` e ganhou uma suíte de testes que exercita os 202 endpoints com login real e banco real. A área de administração foi fechada para quem não tem `ROLE_ADMIN`.

No caminho foram corrigidos vários problemas:

- enum lido e escrito pela descrição;
- `LazyInitializationException`;
- falhas de isolamento por empresa;
- uma injeção de JPQL que permitia roubar a sessão do administrador;
- a resposta do `JwtFilter` para token inválido.

As decisões tomadas hoje e as que ainda dependem de definição estão registradas em `docs/pendencias.md`.

9 → 337
testes, todos passando

202
endpoints cobertos por teste de API

114
arquivos: 66 alterados, 48 novos

0
commits: nada foi enviado ao git

Principais entregas

Tema	Resultado
DTO em toda saída	11 DTOs novos e 3 ajustados; nenhum controller devolve entidade JPA.
Enum no JSON	A causa do erro ao salvar era o Jackson 3, que usa o `toString()`. Agora o enum vai e volta sempre pelo nome.
open-in-view desligado	Desligado nos perfis dev e prod. A transação fica no service, e a conversão para DTO acontece dentro dela.
Isolamento por empresa	5 falhas corrigidas: as 4 já documentadas e o `buscar-por-login`.
Token recusado (P7)	Responde 401 com o motivo. O login voltou a funcionar com um token velho no header.
Entidades	`toString`, `equals` e `hashCode` não tocam em relacionamento LAZY nem em dado sensível.
Área de administração (P9, P10, P21)	50 endpoints são só de `ROLE_ADMIN`, e a injeção de JPQL pelo `sort` foi fechada.
Regras alinhadas ao banco (P11, P13, P15, P20)	Plano FREE; CPF, CNPJ e e-mail obrigatórios; preço e estoque com mínimo 1; data de cadastro preenchida pelo back-end; arquivo da mensagem obrigatório.
Testes	Testes de API com login real, testes com sessão real do Hibernate e testes unitários de service. Nenhum resíduo fica no banco dev.

O que muda para o front-end

- Enum vai e volta pelo nome (`CARTAO_CREDITO`), não pela descrição.
- Todas as respostas são DTO: relacionamentos chegam como `xxxId` + `xxxNome`.
- Usuário sem `ROLE_ADMIN` recebe 403 nos seguintes endpoints:
  - planos, roles e atribuição de roles;
  - listagem e pesquisa de empresas;
  - cadastro, edição e exclusão de usuários;
  - `deletarTodos` e `deletarTodosPorIds`.
- Token inválido, vencido ou de sessão substituída responde 401 com o motivo na `message`.
- CPF, CNPJ e e-mail da pessoa são obrigatórios, e a data de cadastro do request é ignorada.
- O `sort` da paginação só aceita o nome de um campo da entidade.

Próximos passos sugeridos

1. Revisar e commitar as mudanças (`/revisar-commit`).
2. Responder as perguntas da P16 (vigência e bloqueio do admin) e da P18 (fórmulas dos totais), e implementar a P14.
3. Confirmar as matrizes de estoque (P17) e de status (P19), refeitas com os enums reais.
4. Definir o papel do módulo `-pro` (P2): nada do que foi feito hoje está nele.

Detalhamento por blocos

Cada bloco traz a situação encontrada, o que mudou, como foi verificado e os arquivos principais. Os números P1 a P21 são as pendências de `docs/pendencias.md`.

1 DTOs na saída dos controllers

Situação

Vários controllers devolviam a entidade JPA. Isso expunha campos internos, acoplava o JSON ao banco e ainda tentava ler relacionamentos LAZY durante a serialização.

O que mudou

- **DTOs novos (11):** Categoria, Chamado, ItemPedido, Mensagem, MovimentacaoProduto, Pedido, Pessoa, Plano, Produto, Role e RoleUsuario.
- **DTOs ajustados:** `ClienteFuncionarioDTO`, `UsuarioDto` e `EmpresaDTO`.
- **Relacionamentos:** viram `xxxId` + `xxxNome`. O nome do usuário vem de `DtoUtil.nomeUsuario`.
- **Conversão:** feita no service, em `converterParaDto` e `converterListaParaDto`. A paginação usa `page.map`.
- **Exceção mantida:** o método usado por outro service continua devolvendo a entidade, e o controller usa uma variante com sufixo `Dto`:
  - `PessoaService.buscarPorId` / `buscarPorIdDto`;
  - `RoleService.buscaPorAcesso` / `buscaPorAcessoDto`;
  - `RoleUsuarioService.salvar` / `salvarDto`;
  - `ClienteFuncionarioService.findByPessoa` / `findByPessoaDto`.

Arquivos: `dto/*`, `util/DtoUtil`, os 14 controllers e os services correspondentes.

2 Enum no JSON

Situação

- **Sintoma:** salvar com `"DINHEIRO"` dava 400, e as consultas devolviam a descrição (`"Cartão de Crédito"`).
- **Causa:** o Jackson 3 (Spring Boot 4) lê e escreve enum pelo `toString()` por padrão. Seis enums sobrescrevem esse método com a descrição: `FormaPagamento`, `StatusChamado`, `PrioridadeChamado`, `TipoChamado`, `TipoPedido` e `TipoMovimentacaoProduto`.
- **DTOs:** alguns tinham o enum declarado como `String`.

O que mudou

- **Configuração:** `config/ConfiguracaoJackson` desliga `READ_ENUMS_USING_TO_STRING` e `WRITE_ENUMS_USING_TO_STRING`.
- **Tipos nos DTOs:** `ClienteFuncionarioDTO` e `UsuarioDto` passaram a usar o tipo enum.
- **EmpresaDTO:** `planoId` passou a `Long`.

3 open-in-view desligado e LazyInitializationException

Situação

- Com `open-in-view=false`, apareciam `LazyInitializationException` em Role, Pessoa, Empresa e ClienteFuncionario.
- `deletarTodos` e `deletarTodosPorIds` davam 500 ("read-only transaction"), com ou sem OSIV. O delete em lote herdava a transação somente leitura do `SimpleJpaRepository`.
- O pool usa `auto-commit=false`, então gravação fora de transação não é confirmada.

O que mudou

- `spring.jpa.open-in-view=false` nos perfis dev e prod.

- Todo service tem `@Transactional(readOnly = true)` na classe e `@Transactional` nos métodos que gravam. A conversão para DTO acontece dentro da transação.
- O usuário logado é carregado no `JwtFilter`, fora da sessão. Quando o DTO precisa do nome dele, o service o busca de novo (ex.: `ChamadoService.salvar`).
- O `@NotNull` de `empresa` saiu de Chamado, ItemPedido, Mensagem, MovimentacaoProduto e Pedido, e também de `Chamado.dataAbertura`. O service preenche esses valores; as colunas continuam `NOT NULL`.

4 Nomes vazios após salvar

Em RoleUsuario, ClienteFuncionario e Empresa, o objeto do request chegava só com o id, e o DTO voltava com o nome `null`. Agora o service troca esse objeto pelo registro do banco antes de gravar (`validarX`, ou `getReference` em Empresa).

5 Isolamento por empresa

Onde	Falha e correção
`CategoriaService.atualizar`	Movia para a empresa logada uma categoria de outra empresa; o teste reproduziu a falha. Agora busca com `buscarPorId(id, empresaId)` e grava o registro existente.
`ClienteFuncionarioService.salvar`	Aceitava pessoa ou usuário de outra empresa. Agora valida os dois na empresa logada.
`RoleUsuarioService.salvar` / `atualizar`	Usuário e registro são conferidos na empresa logada, e o acesso é buscado no banco.
`MensagemService.updateLida`	Confere se a mensagem é da empresa logada.
`/api/usuario/buscar-por-login`	Devolvia usuário de qualquer empresa. Agora responde 404 para login de outra empresa.

6 Outros erros corrigidos

- **Erros 500:**
  - `EmpresaService.buscaPorId` com id inexistente agora responde "Empresa com id: X não foi encontrada.".
  - `UsuarioService.atualizar` e `alterarSenha` chamavam `.get()` sem conferir o resultado; agora respondem com mensagem.
- **EmpresaDTO:** o `pessoaId` recebia o id do plano; agora vêm `pessoaId`, `planoId` e `planoNome` corretos.
- **Usuário:** o `buscarPorId` não trazia `login` nem `pessoaId` e por isso não servia para a tela de edição.
- **`EmpresaService.salvar`:** conferia duplicidade antes de checar se o usuário era admin, e caía em `.get()` sem tratamento. A ordem foi corrigida.

7 Testes de API e autenticação

O que foi criado

- **Base `contexto/TestContextoApi`:**
  - filtros ligados, login real (`POST /api/usuario/login`) e `open-in-view=false`;
  - sem `@Transactional` no teste, de propósito: assim um erro de LAZY aparece;
  - cada classe cria a sua empresa de teste, com um admin, e apaga tudo dela no final, em SQL na ordem das FKs.
- **Cobertura:** 14 classes de controller e `security/TestJwtFilter` cobrem os 202 endpoints; nenhum ficou sem teste.

Verificação

- Antes das correções, a bateria reproduziu os problemas da sessão: enum, "read-only transaction", `LazyInitializationException`, NPE e `NoSuchElementException`, e a falha de isolamento.
- As contagens das tabelas reais do banco dev foram conferidas antes e depois de cada rodada e ficaram iguais.

8 Testes com sessão real do Hibernate

`service/TestSessaoHibernate` tem 16 testes e chama os services direto, sem transação no teste, com o usuário montado como no `JwtFilter`:

- **Ambiente:** o primeiro teste confirma que não há sessão aberta e que ler um LAZY fora do service dá `LazyInitializationException`. Isso mostra que o teste consegue detectar o erro.
- **Services:** 12 services gravam e leem, e o teste confere os nomes vindos de relacionamentos nos DTOs.
- **Entidades:** 3 testes leem as entidades fora da sessão e conferem `toString`, `equals` e `hashCode`.

Prova de que o teste funciona

Tirei o `@Transactional` do `ClienteFuncionarioService` e o teste quebrou com `LazyInitializationException`. Depois restaurei o arquivo e conferi que ficou idêntico ao original.

9 Token recusado responde 401 P7

Situação

- O `JwtFilter` lançava exceção para token inválido, vencido ou de sessão derrubada; num servidor real, isso tende a virar erro 500.
- O `JwtService.validarToken` lançava exceção em vez de devolver `false`. Por isso o próprio login falhava quando o front mandava um token velho no header. Esse bug apareceu nos testes.

O que mudou

- **Token recusado:** a requisição segue sem autenticação e o motivo fica guardado. Numa rota protegida, o `JwtAuthenticationEntryPoint` responde 401 com esse motivo; o login continua aberto.
- **Erro inesperado** (ex.: banco fora do ar): responde 500 pelo `GlobalExceptionHandler`.
- **Limpeza:** saíram o `printStackTrace` e o `validarToken`. O log registra o motivo e a rota, nunca o token.

Caso	Mensagem do 401
Sem token	Usuário não autenticado ou token inválido
Mal formado ou assinado por outra chave	Token de acesso inválido.
Vencido	Token de acesso expirado, faça login novamente.
Um login novo substituiu a sessão	Sessão encerrada ou substituída por um novo login, faça login novamente.
Usuário ou empresa bloqueados depois do login	As mesmas mensagens do login

Testes: `TestJwtFilter` (15, fluxo real) e `TestJwtFilterUnitario` (6, Mockito, inclusive o caso de banco fora do ar).

10 Entidades seguras fora da sessão

Situação

- O `@Data` do Lombok gerava `toString`, `equals` e `hashCode` que liam os relacionamentos LAZY. Fora da sessão, as 12 entidades com relacionamento quebraram no teste.
- O `toString` também incluía senha, tokens, CPF, CNPJ e e-mail.

O que mudou

- **Lombok:** `@Getter @Setter @ToString(onlyExplicitlyIncluded = true)`. O `toString` lista só os campos simples permitidos.
- **equals e hashCode:** vêm da nova `model/EntidadeBase`.
  - Comparam pelo id.
  - São `final` e funcionam com proxy, mesmo sem sessão. O `Hibernate.getClassLazy` exige sessão no Hibernate 7, por isso não foi usado.
  - Isso vale porque o projeto não usa herança entre entidades.

Testes: `TestEntidadeBase` (6) e 3 em `TestSessaoHibernate`.

11 Área de administração P9 P10

Decisão: a permissão fica nos dois lugares.

- **Controller:** todo controller tem `@PreAuthorize`, com `isAuthenticated()` para dados da empresa e `hasAuthority('ROLE_ADMIN')` para a área admin.
- **Service:** confere `isAdmin()` e responde 403.
- **Erro 403:** o `GlobalExceptionHandler` passou a tratar `AccessDeniedException`; sem isso, a negação viraria 500.

Brecha encontrada	Como ficou
Qualquer usuário se dava `ROLE_ADMIN` em `/api/roleUsuario/salvar`	`/api/roleUsuario` só para admin
Qualquer usuário criava, renomeava ou apagava roles. Renomear `ROLE_USER` para `ROLE_ADMIN` tornaria todos admin.	`/api/role` só para admin
Qualquer usuário alterava ou apagava planos, que valem para todas as empresas	`/api/plano` só para admin
Qualquer usuário listava e pesquisava as empresas de todos os clientes	Só o admin; o usuário comum vê só a própria empresa
Usuário comum apagava usuários (`deletar/{id}`, `deletar-todos`)	Só o admin
`alterarSenha` aceitava o id de qualquer usuário	O usuário comum só troca a própria senha
Injeção de JPQL no `sort` da paginação: dava para ler o token de sessão do admin e entrar como ele	Só aceita o nome de um campo simples da entidade; o resto responde 400
`EmpresaService.salvar` revelava se uma pessoa já tinha empresa antes de checar a permissão	A permissão é a primeira verificação

Testes: `security/TestPermissoesAdmin` (56 testes, incluindo os 50 endpoints de admin chamados por um usuário comum) e os testes unitários de service.

12 Regras alinhadas ao banco e exclusão em lote

Nenhum script SQL foi necessário: o Java passou a seguir as restrições que o banco já tinha, e o banco não mudou.

Pendência	Decisão	Implementação
P11 Plano FREE	O FREE é inserido direto no banco; não se cadastra outro	A API recusa outro FREE, ou transformar um plano em FREE. O valor segue o CHECK do banco: FREE com 0, os demais de R$ 49 a R$ 200.

P12 Unicidade	Únicos no banco inteiro	Já era assim; nada mudou.
P13 CPF e CNPJ	Seguir as restrições do banco	CPF, CNPJ e e-mail obrigatórios (`@NotBlank`), com mensagem clara.
P15 Preço e estoque	Seguir o CHECK do banco: no mínimo 1	`@NotNull` e mensagens corrigidas: diziam "maior que 1" para um mínimo de 1.
P20 Datas e anexo	Seguir as restrições do banco	A data de cadastro da pessoa é preenchida pelo back-end e mantida na edição. O arquivo da mensagem é obrigatório.
P21 Exclusão em lote	Só admin	`deletarTodos` e `deletarTodosPorIds` de 7 módulos, conferidos no controller e no service.

Efeito para a P17

Com o estoque mínimo de 1 (P15), o estoque nunca chega a 0. Quando a baixa automática existir, vender a última unidade vai dar erro. Isso está registrado na P17.

13 Documentação e regras do projeto

- **`CLAUDE.md`:**
  - open-in-view desligado e regra de transação;
  - enum pelo nome;
  - entrada como entidade e saída como DTO;
  - `@PreAuthorize` e a área admin;
  - dicas de teste e tempo da suíte.
- **`.claude/rules`:** controllers, services (transação e DTO), testes (bases e padrões), entidades (`EntidadeBase` e `@ToString`) e segurança (área admin, 403 e `sort`).
- **`docs/`:** `regras-negocio.md`, `arquitetura.md` (fluxo do JWT e tabela de 401), `analise-projeto.md` (dívidas resolvidas e testes) e `pendencias.md`.

14 Registrado para depois

Tudo abaixo está em `docs/pendencias.md`, com as perguntas que faltam responder.

Item	Situação	O que falta
P14	Decidida, a implementar	Usuário opcional na edição de ClienteFuncionario.
P16	Decidida, com perguntas	Bloquear acesso com plano inativo ou vencido. Falta definir: • o que significa vigência vazia; • se o dia do vencimento ainda vale; • se o admin também é bloqueado; • se os limites do plano impedem cadastro novo.
P18	Decidida, com perguntas	Totais calculados no back-end. Falta definir as fórmulas, o preço unitário do item e se o desconto é em valor ou percentual.
P17	Aguardando confirmação	Classificação do estoque refeita com o enum real. Faltam: • campo de estoque reservado; • fluxo de compra; • tratamento do cancelamento; • a questão do estoque mínimo de 1.

P19	Aguardando confirmação	Matrizes de status de Pedido e Chamado refeitas com os enums reais; histórico de status.
P1–P6, P8	Ambiente	IDE, papel do `-pro`, perfil prod, execução do `V1_INIT.sql`, serviço PostgreSQL 9.5, `spring.security.user`, versão do jsr310.

Dívidas técnicas ainda abertas (`analise-projeto.md`)

- CORS aberto (`*` com credenciais) e segredos em texto nos `.properties` e no histórico do git.
- `ddl-auto=update` sem ferramenta de migration; o perfil prod é cópia do dev.
- Consultas N+1 nas listagens com nome de usuário.
- Status HTTP inadequados: `ConstraintViolationException` devolve 500, e criar devolve 200 em vez de 201.
- Sem `@Version` nas entidades.
- Os testes gravam no banco dev, numa empresa descartável e com limpeza no final.

Apêndice A — Evolução dos testes

Testes	O que entrou
9	Situação no início da sessão: só testes de repository de categoria e o contexto.
196	`TestContextoApi`, 14 classes de controller, `TestJwtFilter` e 5 classes de testes unitários de service (isolamento por empresa).
229	`TestSessaoHibernate`, `TestEntidadeBase`, `TestJwtFilterUnitario` e os casos de 401 no `TestJwtFilter`.
293	`TestPermissoesAdmin` e testes unitários de Plano, Role, Empresa, Usuario e RoleUsuario.
337	Exclusão em lote (P21), plano FREE (P11) e validações alinhadas ao banco (P13, P15, P20).

Apêndice B — Principais arquivos novos

Arquivo	Para que serve
`config/ConfiguracaoJackson`	Enum sempre pelo nome no JSON.
`model/EntidadeBase`	`equals` e `hashCode` pelo id, seguros com proxy LAZY.
`dto/*DTO` (11 classes)	Saída de todos os controllers.
`util/DtoUtil`	Nome do usuário no DTO, sem risco de NPE.
`contexto/TestContextoApi`	Base dos testes de API com login real e empresa descartável.
`service/TestSessaoHibernate`	Garante que não há `LazyInitializationException`.
`security/TestPermissoesAdmin`	Garante que um usuário comum não entra na área admin.
`security/TestJwtFilterUnitario`	Caminhos do filtro que não dá para provocar de ponta a ponta.

Apêndice C — Como conferir

- **Suíte inteira**, a partir de `jdev-erp-pro-back-end-dev/`, com o JDK 25 no `JAVA_HOME`: `mvn "-Dmaven.compiler.proc=full" clean test`.
- **Um teste só:** `-Dtest=TestPermissoesAdmin`.
- **Revisão antes do commit:** `/revisar-commit`.

Antes de commitar

Os testes alteram os arquivos de log versionados do módulo (`jdev-erp-pro-back-end-dev.log` e o `.gz` rotacionado). Depois de cada rodada, eles foram restaurados com `git checkout --`. Confira o `git status` antes do commit.